NoSQL Databases

Types, Advantages, and Real-World Applications

Understanding Modern Database Systems

What is NoSQL?

NoSQL Definition

NoSQL (Not Only SQL) refers to non-relational databases designed for:

- Scalability
- Flexibility
- High performance
- Handling unstructured data

Key Characteristics

- Schema-less or flexible schema
- Horizontal scaling (sharding)
- Distributed architecture
- High availability
- Eventual consistency

Types of NoSQL Databases



Document Databases

Store data as documents (JSON, BSON, XML)

Examples:
MongoDB, CouchDB



Key-Value Stores

Simple key-value pairs for fast lookup

Examples:
Redis, DynamoDB



Column-Family Stores

Data stored in columns rather than rows

Examples:
Cassandra, HBase



Graph Databases

Nodes and edges for relationship data

Examples:
Neo4j, Amazon Neptune

Document Database: MongoDB Example

Sample Data: User Profile

```
{
  "_id": "user123",
  "name": "John Doe",
  "email": "john@example.com",
  "age": 30,
  "address": {
    "city": "New York",
    "zipcode": "10001"
  },
  "interests": ["coding", "music"],
  "created_at": "2024-01-15"
```

Key Features

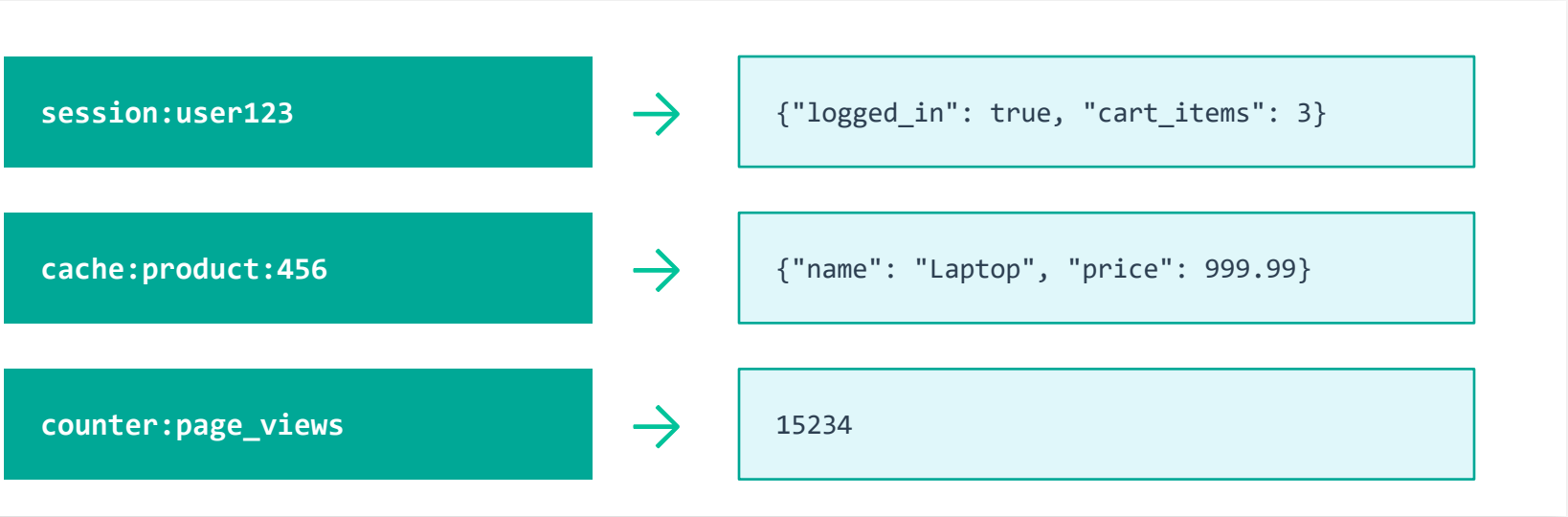
✓ **Flexible Schema**
Documents can have different fields

✓ **Nested Data**
Complex hierarchies in one document

✓ **Arrays**
Store multiple values directly

Key-Value Store: Redis Example

Sample Data: Session Cache



Ultra-fast lookups (O(1)) • Perfect for caching • In-memory storage • Simple but powerful

Column-Family Store: Cassandra Example

Sample Data: Time-Series Sensor Data

Row Key	timestamp	temperature	humidity	pressure
sensor_01	2024-02-13T10:00:00	23.5°C	65%	1013 hPa

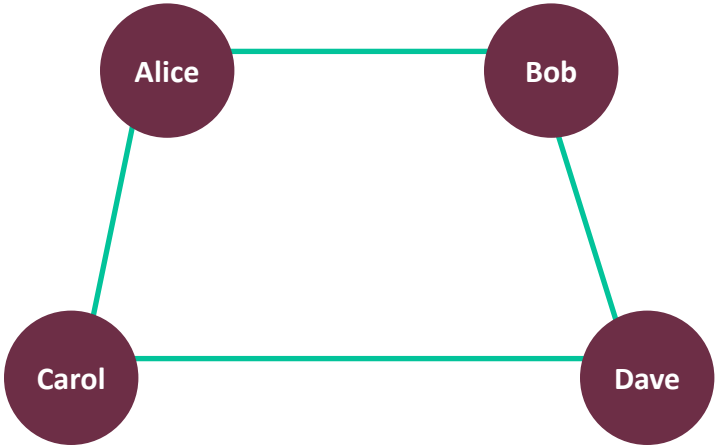
✓ Optimized for write-heavy workloads

✓ Efficient range queries on columns

✓ High scalability and availability

Graph Database: Neo4j Example

Sample Data: Social Network



Perfect For:

- • Social networks
- • Fraud detection
- • Recommendation engines
- • Network analysis
- • Knowledge graphs

The Core Dataset

Entity: A Customer Order

- **Customer:** Alex (ID: 101)
- **Items:** Laptop ($1200), Mouse ($50)
- **Shipping:** 123 Tech Lane, NY.
- **Tags:** "Electronics", "RemoteWork"

Relational (RDBMS)

Philosophy: Structure and Integrity. Data is "normalized" to reduce redundancy.

The Structure

- **Table: Customers** (ID, Name)
- **Table: Products** (ID, Name, Price)
- **Table: Orders** (ID, Customer_ID, Date)
- **Table: Order_Items** (Order_ID, Product_ID, Quantity)

Best For: Complex joins, financial transactions (ACID compliance), and when data consistency is non-negotiable.

NoSQL - Document Store

Philosophy: Storing data exactly as it looks in the application (usually JSON).

The Example (MongoDB Style)

JSON

{ "order_id": "ORD-99", "customer": { "id": 101, "name": "Alex" }, "items": [{"name": "Laptop", "price": 1200}, {"name": "Mouse", "price": 50}], "tags": ["Electronics", "RemoteWork"] }

Best For: Content management, catalogs, and rapid development where schemas change often.

NoSQL - Key-Value Store

Philosophy: Extreme speed. A simple dictionary lookup.

The Structure (Redis Style)

Key	Value (String or JSON)
order:99	{"status": "shipped", "eta": "2 days"}
session:user:101	{"cart_count": 2, "last_login": "14:00"}

Best For: Caching, session management, and real-time leaderboards.

NoSQL - Column-Family (Wide-Column)

Philosophy: Optimized for massive scale and "heavy" writes. Data is grouped by columns rather than rows.

The Structure (Cassandra Style)

Instead of one "Alex" row, you might have:

- **Partition Key:** Customer_ID (101)
- **Columns:** * `order_2023: {items: [...]}`
    - `order_2024: {items: [...]}`

Best For: Time-series data, IoT logs, and write-heavy global applications (like Netflix preferences).

NoSQL - Graph Database

Philosophy: Relationships are "first-class citizens." It’s not about the data point, but how it connects to others.

The Structure (Neo4j Style)

- **(Alex)** —[:PURCHASED]—> **(Laptop)**
- **(Laptop)** —[:CATEGORY]—> **(Electronics)**
- **(Sam)** —[:PURCHASED]—> **(Laptop)**

Best For: Recommendation engines ("Users who bought this also liked..."), social networks, and fraud detection.

Advantages of NoSQL



Scalability

Horizontal scaling by adding more servers. Can handle massive data volumes and traffic.



Flexibility

Schema-less design allows easy adaptation to changing requirements without migrations.



Performance

Optimized for specific use cases. Fast read/write operations with denormalized data.



Cost-Effective

Runs on commodity hardware. Lower total cost of ownership at scale.



High Availability

Built-in replication and fault tolerance. No single point of failure.



Developer Friendly

Data structures map naturally to objects in programming languages.

BASE vs ACID Properties

ACID (SQL Databases)

Atomicity
All operations succeed or all fail

Consistency
Data always in valid state

Isolation
Concurrent transactions don't interfere

Durability
Committed data persists

BASE (NoSQL Databases)

Basically Available
System appears to work all the time

Soft State
Data can be in inconsistent state temporarily

Eventually Consistent
Data becomes consistent over time

BASE: Advantages & Trade-offs

✓ Advantages

- • Better scalability
- • Higher availability
- • Better performance
- • Partition tolerance
- • Works in distributed systems

⚠ Trade-offs

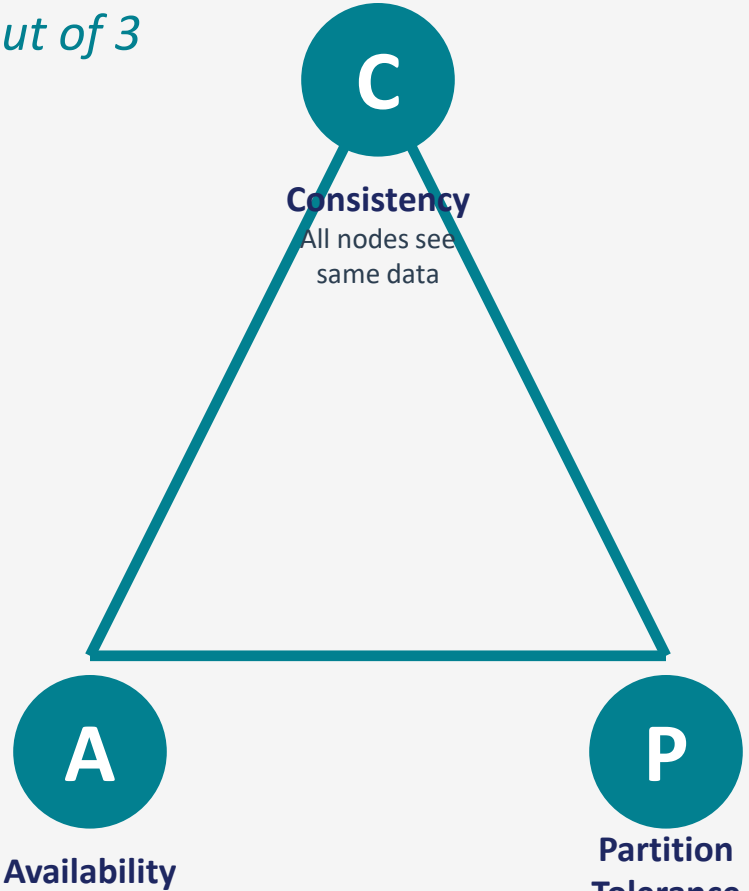
- • Temporary inconsistency
- • No ACID guarantees
- • Complex conflict resolution
- • Application handles consistency

Real Example: Social Media 'Likes'

When you like a post, different users might see different counts temporarily. **Eventually, all users see the correct count.** This trade-off allows Facebook/Instagram to handle billions of likes per day without system slowdown.

CAP Theorem

You can only choose 2 out of 3



CAP Theorem: Real-World Example

E-commerce Shopping Cart Scenario

Scenario: Your database has 3 servers (A, B, C) across different regions. Network partition occurs between server A and servers B+C.

CP: Consistency + Partition Tolerance

Sacrifice Availability • When partition occurs, system becomes unavailable to maintain consistency. Users see error: 'Service temporarily unavailable.' *Ex: Banking systems, financial transactions*

AP: Availability + Partition Tolerance

Sacrifice Consistency • System stays available but may show stale data. User adds item to cart on server A, but server B doesn't see it yet. *Ex: Social media, shopping carts, content delivery*

CA: Consistency + Availability

Sacrifice Partition Tolerance • Works only in single datacenter with no network partitions. Not practical for distributed systems. *Ex: Traditional single-server databases*

NoSQL in Industry

Netflix

Database: Cassandra

Stores viewing history, user preferences, and metadata for 200M+ subscribers

Scale: *Billions of writes/day*

Facebook

Database: MongoDB & Cassandra

Social graph, messaging, newsfeed data for 3B+ users

Scale: *Petabytes of data*

Uber

Database: Redis & DynamoDB

Real-time driver location tracking, surge pricing, trip data

Scale: *Millions of requests/sec*

LinkedIn

Database: Voldemort (Key-Value)

Profile data, connections, recommendations

Scale: *1B+ profiles*

Twitter

Database: Manhattan & Redis

Timeline generation, tweet storage, trending topics

Scale: *500M tweets/day*

Amazon

Database: DynamoDB

Shopping cart, product catalog, order processing

Scale: *20M requests/sec*

SQL vs NoSQL: Head-to-Head

Feature	SQL (Relational)	NoSQL
Data Model	Tables with fixed schema	Flexible (documents, key-value, etc.)
Schema	Predefined, rigid	Dynamic, schema-less
Scaling	Vertical (add more power)	Horizontal (add more servers)
ACID/BASE	ACID guarantees	BASE (eventual consistency)
Relationships	JOINs supported	Denormalized or references
Query Language	SQL (standard)	Database-specific APIs
Best For	Complex transactions, reporting	Big data, real-time, flexibility
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Cassandra, Redis

What is NewSQL?

Best of Both Worlds

NewSQL combines:

- SQL's ACID guarantees and relational model
- NoSQL's scalability and performance

SQL Features

- ACID transactions
- SQL queries
- JOINs
- Strong consistency
- Relational model

NoSQL Features

- Horizontal scaling
- Distributed architecture
- High performance
- Cloud-native
- Auto-sharding

Examples

- Google Spanner
- CockroachDB
- VoltDB
- NuoDB
- Amazon Aurora

Complete Comparison: SQL vs NoSQL vs NewSQL

Aspect	SQL	NoSQL	NewSQL
Data Model	Relational (tables)	Various (doc, KV, graph)	Relational (tables)
Schema	Fixed, predefined	Flexible, dynamic	Fixed, predefined
Transactions	Full ACID	Limited or BASE	Full ACID
Scalability	Vertical	Horizontal	Horizontal
Consistency	Strong	Eventual (typically)	Strong
Query Language	SQL	Varied APIs	SQL
Performance	Good for complex queries	Optimized for specific use	High performance at scale
Use Cases	Traditional apps, ERP	Big data, real-time	Global scale + ACID needs
Learning Curve	Moderate	Varies	Moderate to High
Maturity	Very mature	Mature	Emerging
Examples	PostgreSQL, MySQL	MongoDB, Cassandra	CockroachDB, Spanner

Decision Guide: When to Use What?

Use SQL When...

- Data is structured and relationships are important
- ACID compliance is critical (banking, finance)
- Complex queries with JOINs are frequent
- Data integrity is paramount
- Mature tooling and support needed

Use NoSQL When...

- Handling massive volumes of data
- Schema flexibility is required
- Horizontal scaling is necessary
- Working with semi-structured or unstructured data
- High-speed reads/writes needed (real-time apps)

Use NewSQL When...

- Need both ACID and scalability
- Global distribution with strong consistency
- Complex transactions at massive scale
- Want SQL but need NoSQL performance
- Building modern cloud-native applications

Same Data, Different Storage

E-commerce Order: Stored in 4 Different NoSQL Types

Original Order: Order #12345 | Customer: Alice | Items: [Laptop $999, Mouse $25] | Total: $1024 | Date: 2024-02-13

Document DB (MongoDB)

```
{
  "order_id": "12345",
  "customer": "Alice",
  "items": [
    {"name": "Laptop", "price": 999},
    {"name": "Mouse", "price": 25}
  ],
  "total": 1024,
  "date": "2024-02-13"
}
```

Nested structure, all data in one document

Column-Family (Cassandra)

```
Row Key: 12345
Customer: Alice
Item_1_name: Laptop
Item_1_price: 999
Item_2_name: Mouse
Item_2_price: 25
Total: 1024
Date: 2024-02-13
```

Flattened into columns, efficient for queries

Key-Value (Redis)

```
order:12345 →
'{"customer":"Alice","items":
[{"name":"Laptop","price":999},
{"name":"Mouse","price":25}],
"total":1024,"date":"2024-02-13"}'
```

Simple key-value pair, value as JSON string

Graph DB (Neo4j)

```
(Order:12345)
  -[:PLACED_BY]→(Customer:Alice)
  -[:CONTAINS]→(Item:Laptop)
  -[:CONTAINS]→(Item:Mouse)
  [:TOTAL]→1024
  [:DATE]→2024-02-13
```

Relationships as first-class citizens

Key Takeaways

- NoSQL databases offer flexibility, scalability, and performance for modern applications
- Four main types: Document, Key-Value, Column-Family, and Graph databases
- BASE model trades consistency for availability and partition tolerance
- CAP theorem: You can only guarantee 2 out of 3 (Consistency, Availability, Partition Tolerance)
- NewSQL bridges the gap between SQL and NoSQL, offering both ACID and scalability
- Choose the right database based on your specific use case, data structure, and requirements

Thank You

Questions?